Portfolio

ID623002: Introductory Game Development

Assessment Information

Level	Credits	Assessment Type	Weighting
6	15	Individual	100%

Assessment Overview

In this **individual** assessment, you will develop **three** games. In addition, marks will be allocated for code quality and best practices, documentation and **Git** usage.

Learning Outcome

At the successful completion of this course, learners will be able to:

1. Design and build usable, attractive games using various introductory algorithms following an appropriate software development methodology.

Assessments

Assessment	Weighting	Due Date	Learning Outcome
Portfolio	100%	26 June at 4.59 PM	1

Conditions of Assessment

You will complete this assessment during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **26th June** at **4.59 PM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** over all assessments in **ID623002: Introductory Game Development**.

Submission

You **must** submit all application files via **GitHub Classroom**.

- Repository URL: https://classroom.github.com/a/tSkpt5Ho
- Late Penalty: 10% per day, rolling over at 5.00 PM

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using an **AI generative tool**. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

AI Tools

Learning to use AI tools is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

This also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at https://www.op.ac.nz/about-us/governance-and-management/policies.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted for poor time management or pressure of other assessments.

Resits

Resits and reassessments **are not** applicable in **ID623002: Introductory Game Development**.

Additional Information

- **Do not** rewrite your **Git** history. It is important that the course lecturer can see how you worked on your assessment over time.
- You need to show the course lecturer the initial **GitHub** issues before you start your development work. Following this, you need to show the course lecturer your **GitHub** issues at the end of each week.

Functionality - Learning Outcome 1 (50%)

Breakout - 15%

- Build and publish the game to **itch.io**.

Fundamental Mechanics

- **Main menu** - The game should have a main menu with options to start the game, view the high scores and exit the game.
- **Paddle movement** - The player should be able to move the paddle left and right using the **A** and **D** keys using the **Unity** input system.
- **Ball movement.**
- **Collisions** - The ball should bounce off the walls, paddle and bricks.
- **Brick formation** - The game should have a 10 x 4 grid of bricks.
- **Brick health** - Each brick should have a health value that determines how many hits it can take before being destroyed.
- **Score system** - One point should be awarded to the player who destroys a brick.
- **Score display** - The score of the player should be displayed on the screen.
- **Live system** - The player should have **3** lives.
- **Lives display** - The lives of the player should be displayed on the screen.
- **High scores** - The game should store and display the top **10** high scores.
- **Sound effects** - The game should have sound effects for the ball bouncing off the walls, paddle and bricks, scoring a point and the game ending.
- **Pause and resume** - The player should be able to pause and resume the game using the **P** key.
- **Game over** - The game should end when the player destroys all the bricks or loses all their lives.
- **End game menu** - The game should have an end game menu with options to play again, view the high scores and exit the game.

Tier 2 Mechanics

Select **three** mechanics from the list below:

- **Difficulty levels** - Easy, medium and hard difficulty levels that adjust the speed of the ball and health of the bricks.
- **Ball speed** - The speed of the ball should increase after a certain number of goals.

- **Countdown timer** - A countdown timer that starts the game after a certain number of seconds.
- **Customisation** - Asset options for the paddle and ball.
- **Music** - Background music that plays during the game.
- **Moving bricks** - Bricks that move horizontally.
- **Obstacles** - Obstacles that block the ball's path.

Tier 3 Mechanics

Select **two** mechanics from the list below:

- **Time limit** - A time limit to complete the game.
- **Three levels** - Three levels with different brick formations.
- **Sound settings** - Options to adjust the volume of the sound effects and music.
- **Brick repair** - Bricks that regenerate health over time.
- **Animated effects** - Animated destruction effects for the bricks.
- **Power-ups** - Random drops that allow the player to shoot multiple balls, increase the size of the paddle and slow down the ball.

Your Own Mechanics

Implement **three** mechanics of your choice. These mechanics should be different from the ones listed above.

RogueLike - 20%

- Build and publish the game to **itch.io**.

Fundamental Mechanics

- **Main menu** - The game should have a main menu with options to start the game, view the high scores and exit the game.
- **Player movement** - The player should be able to move using the **Unity** input system.
- **Enemy movement** - The enemies should move towards the player when they are in range.
- **Combat** - The player and enemies should be able to attack each other.
- **Health system** - The player and enemies should have health values that decrease when attacked.
- **Health display** - The health of the player and enemies should be displayed on the screen.
- **Score system** - One point should be awarded to the player who defeats an enemy.
- **Score display** - The score of the player should be displayed on the screen.
- **Pickup system** - The player should be able to pick up items that increase their health and damage.
- **High scores** - The game should store and display the top **10** high scores.
- **Sound effects** - The game should have sound effects for the player attacking, the enemies attacking, the player being attacked and the game ending.
- **Pause and resume** - The player should be able to pause and resume the game using the **P** key.
- **Game over** - The game should end when the player defeats all the enemies or the player's health reaches zero.
- **End game menu** - The game should have an end game menu with options to play again, view the high scores and exit the game.

Tier 2 Mechanics

Select **three** mechanics from the list below:

- **Difficulty levels** - Easy, medium and hard difficulty levels that adjust the health and damage of the enemies.
- **Item variety** - Different items that provide unique effects to the player.
- **Customisation** - Asset options for the player, enemies and items.
- **Music** - Background music that plays during the game.
- **Random generation** - Randomly generated levels that change each playthrough.
- **Events** - Random events that occur during the game.
- **Multiple levels** - Multiple levels with different enemy types and layouts.

Tier 3 Mechanics

Select **two** mechanics from the list below:

- **Boss battles** - Boss enemies that have unique attack patterns and abilities.
- **Animated effects** - Animated effects for the player attacking, the enemies attacking and the game ending.
- **Sound settings** - Options to adjust the volume of the sound effects and music.
- **Fog of war** - A fog of war effect that hides parts of the map until the player explores them.
- **Minimap** - A minimap that shows the layout of the level and the player's position.
- **Teleportation** - Teleportation pads that allow the player to move between different parts of the level.

Your Own Mechanics

Implement **four** mechanics of your choice. These mechanics should be different from the ones listed above.

Platformer - 15%

- Build and publish the game to **itch.io**.

Fundamental Mechanics

- **Main menu** - The game should have a main menu with options to start the game, view the high scores and exit the game.
- **Player movement** - The player should be able to move left and right and jump using the **A**, **D** and **Space** keys using the **Unity** input system.
- **Gravity** - The player should fall when not on a platform.
- **Collisions** - The player should collide with platforms, walls, enemies and collectibles.
- **Platforms** - The game should have a variety of platforms for the player to navigate.
- **Enemy movement** - Enemies should patrol platforms and damage the player on contact.
- **Score system** - Points should be awarded to the player for collecting items and defeating enemies.
- **Score display** - The score of the player should be displayed on the screen.
- **Live system** - The player should have **3** lives.
- **Lives display** - The lives of the player should be displayed on the screen.
- **High scores** - The game should store and display the top **10** high scores.

- **Sound effects** - The game should have sound effects for jumping, landing, collecting items, defeating enemies and the game ending.
- **Pause and resume** - The player should be able to pause and resume the game using the **P** key.
- **Game over** - The game should end when the player completes all levels or loses all their lives.
- **End game menu** - The game should have an end game menu with options to play again, view the high scores and exit the game.

Tier 2 Mechanics

Select **three** mechanics from the list below:

- **Difficulty levels** - Easy, medium and hard difficulty levels that adjust the speed of the enemies and the number of hazards.
- **Collectibles** - Items scattered across levels that the player can collect for bonus points.
- **Moving platforms** - Platforms that move horizontally or vertically.
- **Countdown timer** - A countdown timer that adds urgency to completing each level.
- **Customisation** - Asset options for the player character and environment.
- **Music** - Background music that plays during the game.
- **Parallax scrolling** - A parallax scrolling effect for the background.
- **Checkpoint system** - Checkpoints that save the player's progress within a level.

Tier 3 Mechanics

Select **two** mechanics from the list below:

- **Multiple levels** - Three or more levels with increasing difficulty and different layouts.
- **Boss battles** - A boss enemy at the end of a level with unique attack patterns.
- **Power-ups** - Random drops that temporarily grant the player abilities such as increased speed, invincibility or double jump.
- **Animated effects** - Animated effects for jumping, landing, collecting items and defeating enemies.
- **Sound settings** - Options to adjust the volume of the sound effects and music.
- **Achievement system** - Achievements that reward the player for completing specific tasks.

Your Own Mechanics

Implement **three** mechanics of your choice. These mechanics should be different from the ones listed above.

Code Quality and Best Practices - Learning Outcome 1 (40%)

Project Structure

The codebase should be well-organised with a clear and logical structure that separates concerns and promotes reusability and maintainability.

Naming Conventions

File, class, method, variable, constant and property names should be clear, descriptive and consistent across the codebase.

Documentation and Comments

The codebase should be well-documented using the **XML** documentation format with comments that explain complex logic and clarify the purpose of classes, methods or complex sections.

Code Formatting

Consistent indentation and spacing should be used across the codebase. This includes ensuring proper indentation for code blocks, following a standard format for braces and adding blank lines between sections.

Dead Code

The codebase should not contain unused or redundant files, classes, methods, variables, constants or properties. Keeping unnecessary code around can lead to confusion, clutter and potential bugs down the line.

Performance and Scalability

The codebase should be optimised for performance, using efficient algorithms and data structures to minimise bottlenecks.

Documentation and Git Usage - Learning Outcome 1 (5%)

- Use **GitHub** issues to help you organise and prioritise your development work. The course lecturer needs to see consistent use of **GitHub** issues for the duration of the assessment.
- In your repository **README.md** file, proivide:
  - URLs to the games on **itch.io**.
  - A list of the mechanics you implemented for each game.
- A **.gitignore** file containing the ignored files in this resource - https://raw.githubusercontent.com/github/gitignore/main/Unity.gitignore.
- Your **Git commit messages** should:
  - Reflect the context of each functional requirement change.
  - Be formatted using an appropriate naming convention style.
